

SPLUNK BOTSV2 — THREAT INVESTIGATION WRITE-UP

100 & 200 Series: Insider Threat + Web Application Attack

Report Date	June 07, 2026
Analyst	Amr Khaled Elgendy
Dataset	Splunk BOTSV2 (Boss of the SOC v2)
SIEM Platform	Splunk Enterprise
Log Sources	pan:traffic, stream:HTTP, stream:SMTP, stream:DNS, WinEventLog
Series Covered	100 Series (Insider Threat) 200 Series (Web Attack)

1. Executive Summary

This write-up documents a structured threat investigation performed on the Splunk BOTSV2 dataset. The 100 Series uncovered an insider threat: employee Amber Turing was identified conducting competitor reconnaissance, exfiltrating a proprietary patent document, and installing TOR Browser to anonymize her activity. The 200 Series revealed an external web application attack against brewertalk.com, including a vulnerability scan, SQL injection via the updatexml() function, and a cross-site scripting attack that resulted in session hijacking and a malicious account being created.

2. Series 100 — Insider Threat (Amber Turing)

Q1 — What website domain did Amber visit?

Starting broad with index="botsv2" amber returned 500/500 events — too many to analyse. The investigation approach was to narrow the search using known context: Amber's activity would involve web traffic (stream:HTTP) and her source IP. Using pan:traffic logs, Amber's internal IP was identified as 10.0.2.101. Since Frothly is a beer company and the competitor is also beer-related, the site field was filtered for beer-related domains:

```
index="botsv2" src_ip="10.0.2.101" sourcetype="stream:HTTP" *beer*
| dedup site
| stats count by site
| sort -count
```

Key SPL logic: dedup removes duplicate site entries; stats count by site aggregates visits per domain; sort -count ranks by frequency. One domain stood out clearly.

Answer: www.berkbeer.com

Q2 — What image file showed the CEO's contact info?

Knowing Amber visited berkbeer.com, the investigation narrowed to specific URI paths accessed on that domain. The uri_path field shows the exact resource requested on the web server. Filtering for berkbeer.com within Amber's HTTP traffic and tabling the URI paths:

```
index="botsv2" src_ip="10.0.2.101" sourcetype="stream:HTTP" berkbeer.com  
| table _time uri_path
```

This returned 12 events. One URI path stood out as an image file (.png extension) matching the question criteria.

Answer: /images/ceoberk.png

Q3 — What is the CEO's name?

The CEO's contact image was found. The next step was to identify who Amber emailed at Berkbeer. Email traffic uses SMTP. Since most companies use the same domain for email (e.g. name@berkbeer.com), filtering SMTP logs for berkbeer.com was the logical next step:

```
index="botsv2" sourcetype="stream:smtp" berkbeer.com
```

This returned 6 logs. Reviewing the email content body and HTML source of the email responses revealed the CEO's name embedded in the communication.

Answer: Martin Berk

Q4 — What is the CEO's email address?

Using the same SMTP filter from Q3, the sender and receiver fields in the 6 logs were reviewed. Knowing the CEO's name (Martin Berk), his email address was directly visible in the log fields.

Answer: mberk@berkbeer.com

Q5 — What employee did Amber contact after the CEO?

The SMTP logs were reviewed in timeline order. After the first contact with mberk@berkbeer.com, a second email appeared approximately 5 minutes later from a different Berkbeer sender. This indicated Amber was referred to another employee by the CEO.

Answer: hbernhard@berkbeer.com

Q6 — What file did Amber send to the competitor?

Continuing with the SMTP filter and reviewing outgoing emails from Amber, the attach_filename field was inspected. Email attachments are captured in this field in stream:smtp logs.

```
index="botsv2" sourcetype="stream:smtp" hbernhard@berkbeer.com  
| table _time attach_filename
```

One email contained a proprietary document attachment — a patent file being exfiltrated to the competitor.

Answer: Saccharomyces_cerevisiae_patent.docx

Q7 — What is Amber's personal email address?

Pivoting to Bernard's email address in the SMTP filter, the conversation thread between Amber and hbernhard@berkbeer.com was reviewed. In one email, Bernard asked Amber for her personal email. Amber's reply email contained the answer — however, the content was Base64-encoded. CyberChef was used to decode the content from Base64 to plaintext.

```
index="botsv2" sourcetype="stream:smtp" hbernhard@berkbeer.com
```

The email body was decoded using CyberChef (From Base64 → UTF-8), revealing Amber's personal address.

Answer: ambersthebest@yeastiebeastie.com

Series 100 — MITRE ATT&CK Mapping

Technique ID	Technique	Observed Activity
T1213	Data from Information Repositories	Amber accessed competitor site to gather CEO contact info
T1048	Exfiltration over Alternative Protocol	Patent document exfiltrated via SMTP to competitor
T1567	Exfiltration Over Web Service	Used personal email to bypass corporate monitoring

3. Series 200 — Web Application Attack (brewertalk.com)

Q1 — What TOR Browser version did Amber install?

The question provided the starting filter. Initial attempts with http_user_agent did not yield clear results. Simplifying the filter to key terms from the question produced 125 matching logs, all referencing the same version number. Logs were sorted by time descending to confirm:

```
index="botsv2" tor amber install  
| sort -_time desc
```

Every matching log referenced the same version, confirming the finding.

Answer: 7.0.4

Q2 — What is the public IPv4 of www.brewertalk.com?

HTTP logs only capture internal (private) IP addresses for the destination server. To find the public IP, DNS logs were queried instead — DNS resolution maps domain names to their public IPs. The `host_addr` field in DNS logs contains the resolved IP address:

```
index="botsv2" www.brewertalk.com sourcetype="stream:dns"  
| table host_addr{  
| dedup host_addr{
```

Key insight: always use DNS logs when looking for public IPs of web servers — HTTP logs will only show the internal/NAT'd destination IP.

Answer: 52.42.208.228

Q3 — What IP ran the web vulnerability scan?

A web vulnerability scan generates a high volume of requests from a single source IP to the target server. Filtering HTTP traffic to brewertalk.com and counting requests by source IP identified the scanner — the IP with the highest request count by a significant margin:

```
index="botsv2" www.brewertalk.com sourcetype="stream:http"  
| stats count by src_ip  
| sort -count
```

Answer: 45.77.65.211

Q4 — What URI path was attacked?

Using the attacker IP (45.77.65.211) and the internal server IP identified from the scan (172.31.4.249 — the `dest_ip` with the highest log count in Q3), HTTP requests were filtered to show the targeted URI paths. The `uri_path` field was analysed:

```
index="botsv2" src_ip="45.77.65.211" dest_ip="172.31.4.249"  
| stats count by uri_path  
| sort -count
```

`member.php` had significantly more hits than any other path — consistent with a targeted attack on a specific endpoint. Other PHP paths appeared fewer than 4 times, while `member.php` dominated the request count.

Answer: /member.php

Q5 — What SQL function was abused?

Narrowing the filter to the specific URI path and adding SQL as a keyword to surface form data containing SQL injection payloads. The `form_data` field captures user-submitted input from HTML forms — the injection point:

```
index="botsv2" src_ip="45.77.65.211" dest_ip="172.31.4.249"  
uri_path="/member.php" SQL  
| table _time form_data
```

The form_data field revealed SQL injection payloads. SELECT appeared frequently but is not a function — it is a SQL clause. The actual abused function, repeated across multiple payloads, was updatexml() — a MySQL function commonly abused for error-based SQL injection to extract data from the database via crafted error messages.

Answer: updatexml

Series 200 — MITRE ATT&CK Mapping

Technique ID	Technique	Observed Activity
T1595.002	Active Scanning: Vulnerability Scanning	Web vuln scan from 45.77.65.211 against brewertalk.com
T1190	Exploit Public-Facing Application	SQL injection via updatexml() against /member.php
T1185	Browser Session Hijacking	XSS used to steal session cookie (value: 1502408189)
T1136	Create Account	Rogue account 'klagerfield' created via CSRF abuse

4. Consolidated IOC Summary

Type	Value	Context	Series
IP	10.0.2.101	Amber Turing internal IP	100
Domain	www.berkbeer.com	Competitor site accessed by insider	100
File	Saccharomyces_cerevisiae_patent.docx	Exfiltrated patent document	100
Email	ambersthebest@yeastiebeastie.com	Amber's personal email	100
Email	mberk@berkbeer.com	CEO contact — target of Amber	100
Email	hbernhard@berkbeer.com	Berkbeer employee — Amber contact	100
Image	/images/ceoberk.png	CEO contact image accessed	100
IP	45.77.65.211	Attacker — web scanner & SQLi	200
IP	52.42.208.228	Public IP of brewertalk.com server	200
URI	/member.php	SQLi target on brewertalk.com	200
SQL Func	updatexml()	Error-based SQLi function abused	200
Cookie	1502408189	Stolen session cookie via XSS	200
Username	klagerfield	Malicious account created via CSRF	200

5. Key SPL Queries Reference

Find Amber's web activity by site:

```
index="botsv2" src_ip="10.0.2.101" sourcetype="stream:HTTP" *beer*
| dedup site | stats count by site | sort -count
```

Find URI paths Amber accessed on berkbeer.com:

```
index="botsv2" src_ip="10.0.2.101" sourcetype="stream:HTTP" berkbeer.com
| table _time uri_path
```

Find Amber's SMTP activity:

```
index="botsv2" sourcetype="stream:smtp" berkbeer.com
```

Find TOR installation:

```
index="botsv2" tor amber install | sort -_time desc
```

Find public IP via DNS:

```
index="botsv2" www.brewertalk.com sourcetype="stream:dns"
| table host_addr{} | dedup host_addr{}
```

Identify vulnerability scanner:

```
index="botsv2" www.brewertalk.com sourcetype="stream:http"
| stats count by src_ip | sort -count
```

Find attacked URI path:

```
index="botsv2" src_ip="45.77.65.211" dest_ip="172.31.4.249"
| stats count by uri_path | sort -count
```

Find SQLi function in form data:

```
index="botsv2" src_ip="45.77.65.211" dest_ip="172.31.4.249"
uri_path="/member.php" SQL
| table _time form_data
```

6. Analyst Notes & Investigation Methodology

- **Start broad, narrow progressively:** Begin with index and a known name/keyword. Identify the right sourcetype first, then add IP/field filters. Jumping to specific fields too early causes missed context.
- **pan:traffic for IP discovery:** Palo Alto firewall logs (pan:traffic) include src/dst IPs, ports, app-ID, and session data — ideal for identifying internal IPs of users before pivoting to application-layer logs.
- **HTTP vs DNS for server IPs:** HTTP logs show internal/NAT'd IPs. For public server IPs, always query DNS (stream:dns) using the host_addr{} field.
- **form_data for injection payloads:** In stream:HTTP logs, form_data captures POST request body content — the primary field to inspect for SQL injection and XSS payloads.
- **Volume anomaly = scan:** A single source IP with significantly more requests than any other is a strong indicator of automated scanning activity.
- **CyberChef for encoded content:** SMTP email bodies are often Base64 or UTF-8 encoded. Always attempt decoding in CyberChef when content appears garbled or non-human-readable.